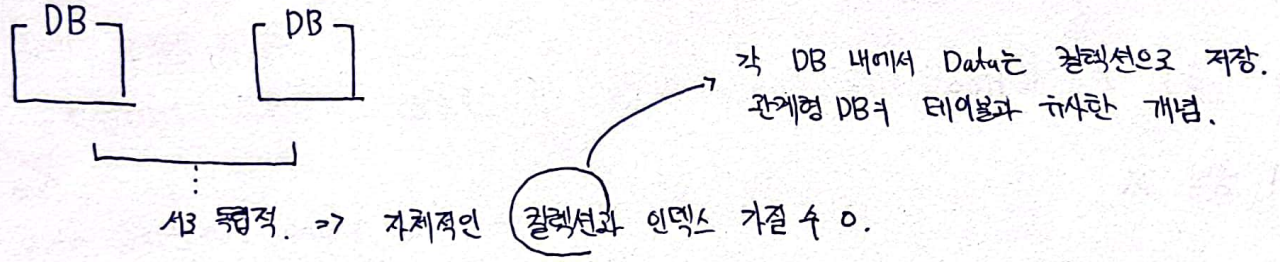
Mongo DB 2. 시작하기

기본 개념.

- Mongo DB 데이터의 기본 단위 => '문서' JSON 형식의 문서로 저장.
- 컬렉션은 스키마 X 인 Table 로 생각.
- Mongo DB의 단일 인스턴스는 여러 독립적인 DB를 호스팅 할 수 있고
  + 각 DB는 자체적인 컬렉션들과 권한 가진다.
- Mongo DB는 Instance 관리와 데이터 조작에 유용한 JavaScript Shell 제공.
- 모든 문서는 문서 컬렉션 내에서 고유한 특수 키 "_id" 가짐.



1) 문서

Mongo DB 핵심 => 정렬된 key + value 의 집합.

ex) JS 객체

{ greeting : "Hello, world!" }

key / value

) 대부분 복잡한 다중 키/값 쌍 가짐

대부분의 경우 키의 순서 중요X.

문서 내 값 => blob (x)

=> 여러 데이터형 중 하나(o)

문서의 key => '문자열' 일부를 제외한 어떤 UTF-8 문자 가능.

조건)

- key는 \u0000 (null문자) 포함 x.
- . , $ 는 몇 가지 특별한 속성 가짐. => 특정한 상황에서 사용.
  보통 예약어 취급하므로 부적절하게 사용 시 드라이버에서 경고.
- _ 로 시작하는 키는 예약어로 생각하는 편.

Mongo DB는 데이터형, 대소문자 구분.

{"foo" : 3} ≠ {"foo" : "3"}

{"foo" : 3} ≠ {"Foo" : 3}

또한 키 중복 x.

2) 컬렉션.

: 문서의 모음.

문서가 RDBMS 에선 '행', 컬렉션에서는 '테이블'로 볼 수 있다.

- 스키마가 없다.

=> 하나의 컬렉션 내 문서들이 모두 다른 구조 가질 수 o.

{"greeting" : "Hello"}
{"foo" : 5}

) 데이터 형과 키가 다르다.
근데 별도의 컬렉션으로 나누는 것이 좋다.

하나 이상의 컬렉션이 필요한 경우.

- 데이터 조직 및 관리.

  : 컬렉션 나누면 더 논리적으로 조직 가능.

  ex)

  User 컬렉션 => 유저 정보 (email, nickname ···)
  games 컬렉션 => 게임 정보 (title, download link ···)
  transactions 컬렉션 => 아이템 구매, 결제 내역.

) 뒤섞여있다면 찾기 힘드니 관리 필요.

- 쿼리 성능 최적화.

  : 같은 컬렉션에 다양한 데이터 섞인다 => 필요 X 인 데이터까지 검색해야해서 성능 ↓

  ∴) 컬렉션 구분
  => 필요 데이터 조회해서 속도 ↑

- 데이터 인덱싱 최적화.

  : 컬렉션마다 다른 인덱스 설정할 수 있어서 검색속도 최적화 가능

  ex)
  User 컬렉션에서 email 필드 기준으로 인덱스 생성.
  game 컬렉션에서 title 필드 기준으로 인덱스 만들면 검색 속도 ↑.

- 데이터 일관성 유지

  : 컬렉션 구분 => 관련 데이터 구조 유지.

  ex)
  User 컬렉션에 password 필드 O
  games 컬렉션에 password 필드 X

) 데이터를 다 섞으면 트랜스퍼리점.

- 확장성 (Sharding & Scaling)

  Mongo DB는 컬렉션 단위로 샤딩 지원.

  Data ↑ => 특정 컬렉션만 분산처리 부하 줄이고 확장가능.

- 네이밍.

: 컬렉션은 이름으로 식별.

ㄴ 빈 문자열 (" ")은 유효한 이름 아님.

ㄴ NO (null 문자)는 컬렉션의 이름의 끝 나타내기에 컬렉션 이름 사용X

ㄴ system. 으로 시작하는 컬렉션 이름X.

시스템 컬렉션에 사용하는 예약어.

ㄴ $ 문자 사용X. 다양한 드라이버들이 시스템에 생성한 컬렉션에서 $문자 사용하기에 $ 컬렉션 이름에 사용X.

서브 컬렉션.

: 서브 컬렉션의 네임 스페이스이 . 문자 썼다.

ex)

[블로그 기능 가진 애플리케이션]

(blog.posts
  blog.authors) : '체계와 목적'

서브컬렉션이 특별한 속성 없이도 여러 MongoDB 도구에서 지침.

데이터 베이스.

Mongo DB는 문서를 컬렉션에 모아둠

+

여러 컬렉션을 DB라 묶다.

Mongo DB 단일 인스턴스 => (여러 DB 호스팅
DB를 완전히 독립적 취급.

'하나의 DB' : 자체적한 파일 + 분리된 파일로 디스크에 저장.

한 애플리케이션의 데이터는 동일한 DB에 저장하는 편.

DB를 나누는 방식 => 하나의 Mongo DB 서버에서 여러 애플리케이션
or
여러 사용자 위해 데이터 저장할 때 쓰면 좋다.

DB 또한 이름으로 식별.

- 빈 문자열 X.
- ''(공백), ., $, /, \, \0 (null문자) 포함 X.
- 모두 소문자
- 최대 64 byte.

=> 실제 DB 이름은 파일 시스템 상에서 파일 된다는 것.

예약된 DB이름.

L admin
: 인증 관점에서 root 데이터베이스.
: admin DB의 사용자 추가 => 모든 DB에 대한 사용 권한 상속 O.

L local
: 절대 복제 X.
: 특정 서버에만 저장하는 컬렉션이 사용.

L config

: Mongo DB를 샤딩설정하는 경우, config DB는 내부적으로 샤드 정보 저장하는데 사용.

컬렉션 저장하는 DB 이름을 컬렉션 이름 앞에 붙이는 것

=> 올바른 컬렉션 이름인 네임스페이스 얻을 수 있음.

ex)

cms : DB. ...., blog.posts : collection.

컬렉

...사용.

cms.blog.posts => 네임스페이스의 길이 < 100 byte.

4. Mongo DB 시작.

Mongo 공식 사이트 → Version : 8.0.6 → 실행 → Run Service as Network
platform : Window x64
package : msi

→ next → 끝.

실행 : C:\Program Files\MongoDB\Server\8.0\bin

mongod.exe → create database
mongos.exe ✓

→ name, color 지정 후 save & Connect.

admin, config, local 뜨면 성공.

윈도우는 데이터 디렉토리 존재여부 / 쓰기 권한 신경x.
시스템으로 자동 실행했기 때문.

∴ DB는 포트 27017번 사용한다

주의)

28017 port => Mongo DB에서 기본 HTTP 상태 페이지 제공. (2.x ~ 3.x)

But, 현재 최신버전 (4.x ~ 8.x)에서는 28017 port 로 http 기본 페이지 열리지 않음!

-28017 port

: 예전 Mongo db는 27017 보다 1000 높은 28017에서 http로 접근 가능한 관리자 페이지 열었음.

localhost: 28017

- 현재.

: Mongo DB 3.6 이후부터 이 기능이 제거

: 이제 Mongo DB는 관리 페이지 대신 (Compass / mongo ch / Atals UI) 사용.

ⅰ) 수동실행.

Sever off: Ctrl + C : 안전하게 서비종료.

else

(Cmd)에서 net stop MongoDB. ⟷ net start MongoDB.

관리자 권한

5. Mongo DB 쉘

∴ 커맨드 라인에서 Mongo DB 인스턴스과 상호작용하는 Java Script 쉘을 함께 제공.

'Shell' ┬ 실행중인 인스턴스 살펴보기
        ├ 간단한 기능 실험
        └ 관리 기능 수행

∴ MongDB 쉘에서 Java Script 코드 실행 가능하다.

다음 줄 ⇒ shift + Enter.

1) MongoDB Client

: 쉘이 시작할 때 Mongo DB 서버의 test 데이터베이스에 연결하고 이 DB 연결을 전역변수 (db)에 할당. 쉘에서는 주로 이 변수 통해 Mongo DB에 접근.

쉘에서 db 치면 test 나옴.

ex)

foobar 이라는 DB 생성.

use foobar 하면, test에서 foobar로 바뀐다.

2) 기본적인 쉘 작업.

: 데이터를 보고 조작하기 위한 CRUD 기본 작업 쉘에서 사용 가능.

- 생성 (Create)

: insert 기능로 문서를 추가.

ex)

```
post = {
        "title": "My Blog Post",
        "content": "Hello world!",
        ⋮
}
```

올바른 Mongo DB 문서.

∴ insert method 통해 컬렉션에 저장 가능.

db.데이터베이스명.insert(post)

DB에 post가 저장되있음.
컬렉션에서 find()로 확인가능.

ex)
db.`DB명`.find()

└ {
  '_id' : ObjectId(' ~ '), -> _id 출현.
  ⋮
}

- 조회 (Read)

: find는 컬렉션 내 모든 문서 반환.
컬렉션 내 한 문서만 보고싶다면 findOne 사용.

findOne() 기준

└ if) 아무조건 없이 찾기
  : 가장 먼저 삽입된 문서 하나만 보여줌.
  : 내부적으로 _id 오름차순과 비슷. (명시적 정렬x)

└ if) 조건O.
  : 먼저 찾은 1개만 반환.
  : 어떤 문서 나오는 지는 인덱스 유무, 내부 저장 순서 등에 달라짐.

find에서는 일치하는 최대 20개의 문서만 화면에 자동으로 표시.

- 갱신 (update)

- 기존 데이터 갱신

db.`컬렉션명`.updateOne(
  {조건},
  { $set : { 수정할 필드 : 값 } }
)

⟶ ⋮ 성공

{
  acknowledge: true,
  insertId : null,
  matchedCount : 1,
  modifiedCount : 1,
  upsertedCount : 0
}
새문서 추가x.

acknowledge => MongoDB가 요청 잘 받았고 실행O. (true)
insertedId => upsert 안했으니 아무 것도 삽입x (null)
matchedCount => 조건에 맞는 문서 1개찾음 (1)
modifiedCount => 그 문서 실제로 수정됨 (1)

- 새로운 key 추가.

db.컬렉션명.updateOne(
{조건},
{$set : {새로운키 : 값}}
)

updateOne vs updateMany

1) updateOne
: 조건에 맞는 첫 번째 문서 1개.
: 특정 문서 1개만 변경 시
: 보통 ... 1개만 처리

2) updateMany
: 조건에 맞는 문서
: 여러 문서 한번에
: 많으면 시간 증가.

+... update() (옛날 방식)
: 예전 명령어. 안정성 문제로 지금도 남아있음.

공식적인 권장 방식 => updateOne, updateMany.

• 삭제.

구 버전 명령어 => remove()
: 조건 x 인 경우 전체 삭제.

현재 : deleteOne, deleteMany, deleteMany({}), findOneAndDelete()
: 1개 삭제
: 여러문서 삭제
: 조건x 전부지움
: 문서 1개삭제 + 문서 반환.

ex)

db.users.remove()

=> DeprecationWarning : Collection.remove() is deprecated.

⋮

참고 용.

remove()는 권장X.

또한 조건 없이 실행하여 에러.

이제는 더이상 기본 값 전체 삭제 허용X.

```
deleteOne(){                    {
    }조건}          =>            acknowledge : true,
}                                 deletedCount : 1
                                }
```

쉘 활용 팁.

쉘에서 help 입력해 도움말 볼 수 있다

=> db.help()

- 쓰기 어려운 이름.

ex)

version 이라는 컬렉션에 접근하려면 db.version 은 DB 함수이기 때문에

db.version 이라 쓸 수 X.

ex)

foo-bar 는 컬렉션 이름으로는 가능.

But, JS에서는 두 변수의 뺄셈.

∴ db.getConnection("foo-bar") 로 컬렉션 얻기.

8. 데이터 형.

=> Mongo DB의 문서과 JSON은 둘 다 JS 객체과 유사.

JSON => 오직 6개의 데이터 형. ∴ 제한적.

Mongo DB는 JSON의 키/값 쌍 유지하며 추가적인 데이터형 지원.

JSON => null, boolean, 숫자, 문자열, 배열 그리고 객체만 지원

Mongo DB =>

+ α.

1) null 형.

null 값과 존재하지 않는 필드 표현하는데 사용.

{"x" : null}

2) boolean

true 와 false 형.

{"x" : true}

3) 32 bit 정수형.

쉘에서 표현 x. Java Script는 64비트 floating형 지원. ∴ 32bit 정수형은 부동소수점형으로 변환.

4) 64 bit 정수형.

쉘에서 표현 x. 쉘에서 이 값을 특별한 내장 문서등에 표현.

5) 64 bit 부동 소수점형.

쉘 내 모든 숫자는 이 데이터형.

{"x" : 3.14} {"x" : 3}

6) 문자열형.

모든 UTF-8 문자열은 문자열형으로 표현 o.

{"x" : "foobar"}

7) 심볼형.

: 쉘에서 지원 X. DB에서 심볼형 발견되면 문자열로 변환한다.

8) 객체ID형.

: 객체 ID형은 문서의 고유한 12byte ID이다.

{"x" : ObjectId()}

9) 날짜형.

: 날짜형은 1970년 1월 1일 이후 시간 흐름을 1/1000 초로 저장.
표준 시간대 저장 X.

{"x" : new Date()}

10) 코드형.

문서는 JS 코드도 포함 X.

{"x" : function() {....}}

11) 이진 데이터형.

: 이진 데이터형은 임의의 Byte 문자열로 쉘에서 조작 X

12) 최대 값 형.

BSON은 가장 큰 값을 나타내는 특별한 데이터형 가짐.
쉘에서는 이것에 대한 데이터형 X.

13) 최소 값 형.

BSON은 가장 작은 값을 나타내는 특별한 데이터형 가짐.

14) undefined

Undefined를 문서 내 사용.

{"x" : undefined}

15) 배열.

값의 집합 or 목록을 배열로 표현.

{"x" : ["a", "b", "c"]}

16) 내장문서

: 부모 문서의 값으로 내장되는 형태. 문서는 문서를 포함 O

{"x" : {"foo" : "bar"}}

null vs undefined

null)

의미: 명시적으로 값 없음.

생성방식: 직접할당

타입: Object

Mongo DB에서 저장 여부: O ex) {"field" : null}

사용용도: 의도적으로 빈 값 줄 때.

undefined)

의미: 값이 정의되지 않음.

생성방식: 선언만하고 값 할당 X.

타입: undefined

Mongo DB 저장 여부: X. 필드 자체가 X.

사용용도: 아직 값 설정되지 않았을 때.

- 숫자.

JS에서는 숫자형이 1가지.

Mongo DB는 3가지 숫자형. (4byte / 8byte 정수형, 8byte float형)

기본적으로 Mongo DB Shell에서는 모든 숫자 double 형 취급.

- 4byte 형 정수를 DB에서 불러온 후, 정수를 내주지 않고 문서 조작해 DB에 저장하더라도 정수형은 부동 소수점형 숫자로 저장.

- 8byte 정수형 숫자 저장 후, Shell에서 보면 숫자 정확하지 않다고 내장 문서로 표현.

Shell에서 8byte 정수가 부동 소수점형으로 근사치를 내장 문서로 보여주는 것 뿐.

1) double 형으로 정확하게 표현할 수 있는 8byte 정수 삽입.

Shell에서 32비트를 정수를 구성하는 높은 자리 수 4byte 과

낮은 자리 수 4byte 나타내는 top과 bottom이라는 두 key를 추가.

왜 그럴까?

Mongo DB는 BSON 이라는 이진 포맷 사용.

BSON과 JSON은 유사하지만 BSON은 타입 구분 명확 + 이진 데이터 다룰수 있음.

이 중 하나가 int 64.

↳ 8byte 정수. 즉, 64비트로 구성.

JS에서 안전하게 표현 가능한 정수 범위

=> $-2^{53}+1$ ~ $2^{53}+1$ 이 범위 외는 오차 발생할 수 O.

↓

Mongo DB 처리법.

top: 상위 32비트 정수

bottom: 하위 32비트 정수.

) : 64bit 정수를 쪼개서 저장하고 표현

ex)

```
:
"MyLong" : {
    "top" : 2
    "bottom" : 42
}
:
```

=> let value = (2 * Math.pow(2, 32)) + 42.

↳ 8589934592 + 42 = 8589934634.

언제 보임?

Mongo DB Shell (mongo)	보일 수 O
Mongosh (신형 shell)	잘 안보임.
Mongo DB compass	NumberLong(...)
bsondump (CLI tool)	보임.
JSON으로 덤프 or 저장.	보일 수 O.

top : 상위 32bit 정수

bottom : 하위 32bit 정수.

=> JS로 표현 불가능한 64 bit 정수 나타내는 내부 포맷.

참고.

Mongo DB 문서에는 표준 BSON 포맷 x.

단지 Mongo DB가 JS과의 type 층을 구현하려고 보여주는 비표준 포맷.

front-end 에 넘겨줄 때 => NumberLong("~").toString()

- 날짜.

: Mongo DB 날짜형 => JS 날짜객체.

새로운 날짜 객체 생성 => new Date()

:!) new 없이 함수 호출

: 날짜 객체가 아닌 문자열 표현.

문자열과 날짜형 서로 맞지 않음. ∴ 삭제, 생성, 조회 쿼리 등 문제 발생.

- 배열.

배열 ┬ 정렬연산 : 리스트, 스택, 큐
└ 비정렬 연산 : 셋(Set)

ex)

{"things" : ["pie", 3.14]}

: 배열은 값으로 다른 종류의 데이터형 포함 o.

Mongo DB가 배열의 구조 이해하고, 배열의 내용을 어떻게 안에서 꺼내어 연산해야하는 지 알고있음.

) pie를 pi 로 변경.

: 배열이 수정되지 않도록 원천적 갱신 차단.

- 내장문서

ex)

```
{
    "name" : "John",
    "address" : {
        "street" : "~",
        "City" : "~",
        "state" : "~~"
    }
}
```

: 내장문서 예시

배열과 마찬가지로 Mongo DB는 내장 문서 구조 이해.

관계형 DB
-> 다른 두 테이블에서 ("people" 과 "address") 분리된 두 행으로 모델링.

Mongo DB
=> person 문서 내 address 문서 바로 내장 가능.

But, 중복 데이터가 많을 수 있음.

관계형 DB 에서 분리된 테이블 "address" 가 있고 주소에 오타 고치려고하면 한번에 가능.

Mongo DB는 각 person의 문서를 모두 찾아 오타 고쳐야함.

- _id와 ObjectIds.

: MongoDB에 저장되는 모든 문서 => "_id" 키 가짐.

키의 값은 어떤 데이터형이라도 가능.
But, ObjectId형이 기본.

: 각 문서가 고유할 수 있도록 모든 문서는 고유한 식별 "_id" 가지고 있어야한다.

i) ObjectId

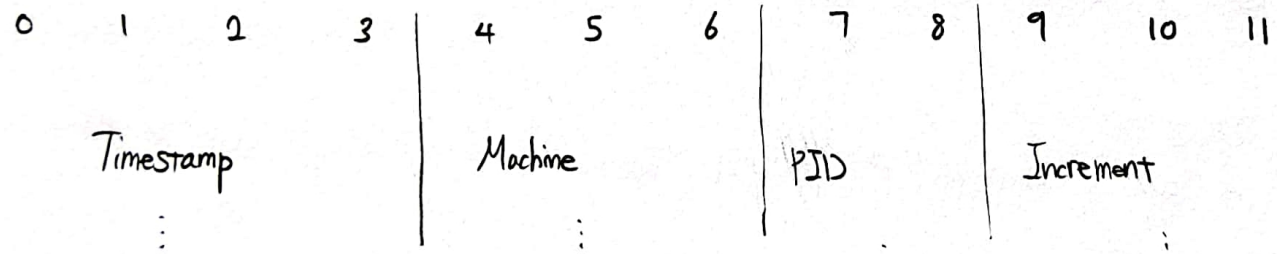
: ObjectId는 "_id"의 기본 데이터형.

↳ 비용 크지 않음 + 전역적으로 고유

ObjectId vs Primary key.

제한된 환경에서 생성 쉬움

다중 서버에 걸쳐 자동증가 PK 동기화 작업 어렵 + 시간 고려 필요.



- 1970.1.1 이후 초 단위
- Timestamp가 먼저 기록되어 대략 입력순서로 정렬.
- 현재시간이 ObjectId에 사용되기 때문에 자주 새로운 값으로 바뀌고 증가.

- ObjectId가 생성된 컴퓨터의 고유 식별자
- 보통 서버 호스트명의 해쉬.
- 각 다른 서버들이 충돌하는 ObjectId를 만들지 못함.

- 프로세스 식별자
- 한 서버에 동시에 생성하는 다른 프로세스 간의 고유성 제공

- 일 초 내 단일 프로세스 고유성 책임자는 증가 카운터.

↳ 첫 9바이트는 일 초 내 서버가 프로세스에 걸쳐 고유성 보장.

ii) _id 자동생성.

: 문서 삽입 시, "_id" 키가 없으면 문서에 자동 추가.

: 일반적으로 클라이언트 측 드라이버가 처리.

iii) 추가적으로...

Mongo DB 같이 분산 확장 용이한 DB도 Client가 분산확장하는 방법이 DB에서 분산 확장하는 방법보다 쉬움.

작업을 클라이언트로 넘겨서 DB 확장 부담↓